Opportunity Forecasting for LLM Search Agents

Anonymous ACL submission

Abstract

Search agents face the decision of whether to continue searching or stop and commit to the best option seen so far at each step. We study how Large Language Models (LLMs) can predict how much more the reward can still improve given a fixed search policy. We generate trajectories with a fixed search model and predict a distribution of the expected remaining reward increase with a forecasting model. We use Monte Carlo continuations at each decision point to train our forecasting models to estimate the distribution of future reward improvement. We then evaluate the forecast on the trajectories in two ways: either stopping a trajectory when the predicted improvement is not worth its cost, or expanding the search thread with the greatest estimated remaining reward increase. We find that in e-commerce and scientific literature search tasks, the trained heads are able to better allocate search budget more effectively than prompting a base LLM. Our results represent how learning expected remaining reward upside can be used as a signal for early stopping in fixed search agents.

1 Introduction

LLM agents that are used to search the web, call the right tools, and interact with a variety of sandboxes and environments face the same recurring question: after identifying a potential candidate, should the agent spend the cost of another step to keep searching or stop and commit to the highest reward option seen so far? (Nakano et al., 2021; Yao et al., 2023; Zhou et al., 2024). We commonly see this exact tradeoff when shopping for items to buy, finding the right papers given an idea, debugging code, and more.

Iterations of this problem have been explored in the past, but they do not provide the formalized control needed by LLM agents. An example are Pandora's box and Weitzman-style models. These begin with a set of alternatives where the reward distributions and cost to view them are already known prior to actually selecting them (Weitzman, 1979). However, when using LLM agents, this becomes a real-time task where what candidates to select in the future are dependent on outputs generated in prior steps. We can represent this by framing it with Partially Observable Markov Decision Process (POMDP). To solve this, we need a transition, observation, and reward model (Smallwood and Sondik, 1973; Lovejoy, 1991). There have been other works in this space that focus on when to stop an LLM agent, but not by estimating how much remaining reward there is in a search space given the searching policy (Liu et al., 2025).

We maintain a fixed search policy to be able to estimate future reward improvement distributions with Monte Carlo continuations and also to isolate the effects and capabilities of the forecasting component. We define a forecasting model in this case as a model that predicts the remaining reward increase given the current search state and the best opened candidate so far in the trajectory. We use this forecast with different stopping and allocation rules to prove its value. We deliberately keep our search policy, and therefore trajectories, fixed as to isolate the quality and value of the forecasts.

We evaluate this formulation in two domains: e-commerce and literature search. We use WebShop for the e-commerce domain as it provides a product-search environment and benchmark (Yao et al., 2022). For the literature search domain, we created Paper Search, which is a literature search environment built from LitSearch and scientific BEIR-style retrieval data (Ajith et al., 2024; Thakur et al., 2021). The rewards for each domain are constructed differently, but everything else like the search, candidate-page lookup, recall, and committing to the best seen so far are the same.

The two main components of our formalization can be seen in Figure 1. The fixed search model

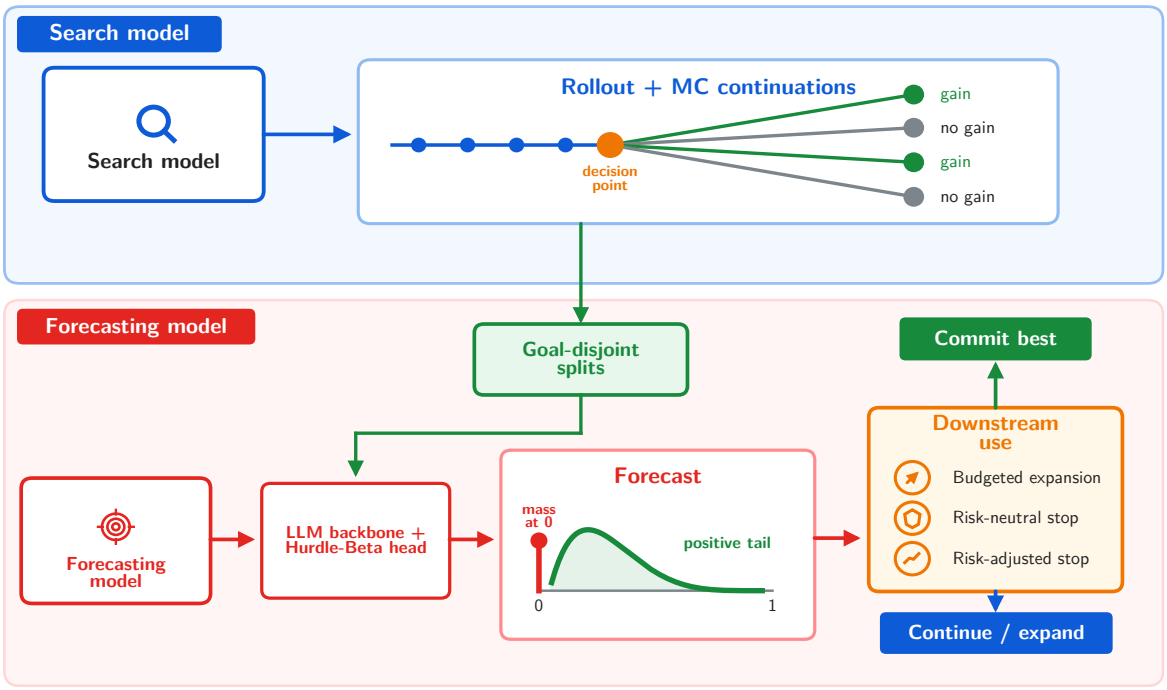


Figure 1: Pipeline for remaining-upside forecasting. Monte Carlo continuations are from decision point and define bounded, zero-inflated remaining-upside labels. A hurdle-Beta regression-head forecaster is used. Its predicted expected reward improvement is used by budgeted expansion across active search threads and per-trajectory stopping policies.

produces trajectories and continuations. Those continuations train the forecaster to estimate the value left in the trajectory. The forecast is then used by stopping rules or by a budgeted-expansion scheduler.

Our contributions are:

- Formulating search-budget control as the expected increase in reward.
- Forecasting a distribution with Monte Carlo continuations and an LLM regression head.
- Reusing one forecasting methodology for per-trajectory stopping and multi-thread budget allocation, and evaluating it against baselines and other trained methods.

2 Related Work

Other related works examine this stopping decision at three main levels: search over a known fixed set of alternatives, general planning or stopping, and prediction with a fixed data generation policy. We discuss all three to ground our assumptions and the role of our formalization.

2.1 Fixed Alternatives: Pandora and Weitzman

A common example of optimal stopping is the secretary problem. In this problem, there is no recall, there is a fixed horizon, there are a fixed number of candidates, and each candidate is generated independently from the same kind of population (Ferguson, 1989). Pandora's Box allows costly inspection with recall, but starts from a fixed set of alternatives (Weitzman, 1979). Extensions of these allow one inspection to update beliefs about other alternatives (Chawla et al., 2021; Gergatsouli and Tzamos, 2023; Atsidakou et al., 2024). When dealing with LLM search agents, the candidates seen in the future depend on the generated output at each step. Using these prior formalizations directly is not possible since the rewards of future candidates do not exist before the candidates are seen.

2.2 General Planning and Agent Termination

Other methods like sequential searching and optimal learning ask when another query is worth its cost (Frazier et al., 2008, 2009; Ke and Villas-Boas, 2019). Budget-aware tool usage and rational meta-reasoning ask related questions as well (Sabbata et al., 2024; Liu et al., 2026). We study the

problem of optimal stopping by only focusing on the forecasting component. Some previous work studies how future states can be determined by current actions. These use POMDPs with stopping actions (Smallwood and Sondik, 1973; Lovejoy, 1991).

Other more recent work has examined how LLM-based agent systems learn to decide which actions to take and when to stop their task. There is some work related to estimating the value of the potential next actions available (Zhai et al., 2025; Koh et al., 2025). Other prior works study how training agents identify when they have gathered enough information to acceptably stop their task (Liu et al., 2025). There is also this study that uses a calibrated approach to understand the compute spent behind the decisions (Park et al., 2025). In our study, we have allocation and stopping rules downstream and use the predicted improvement in reward for those rules. These prior works directly learn to choose search actions or when to stop.

2.3 Calibration and Uncertainty for Language Models

We use some calibration-aware metrics to evaluate our remaining reward improvement. This makes work related to proper scoring rules and predictive calibration relevant (Brier, 1950; Gneiting and Raftery, 2007; Guo et al., 2017). Other more recent work even trains LLMs with a calibration-aware objective rather than only binary correctness (Damani et al., 2026).

3 Problem Setup

3.1 Search domains with recall and search cost

We study two search domains. WebShop text mode provides human-written shopping goals (Yao et al., 2022). Paper Search provides literature-search queries and paper candidates from LitSearch and BEIR scientific IR query, corpus, and query-relevance data (Ajith et al., 2024; Thakur et al., 2021). The agent sees the current page and chooses a valid action, which can be to search or click. After a candidate has been seen and scored, the stopping policy can choose to cut the trajectory short and select the candidate with the best reward we have seen up until that point. We use this as recall. Prior opened candidates can still be chosen when the agent decides to stop. With a fixed search policy π_{search}, stopping chooses a stopping time $\tau \in \mathcal{T}$ under a cost for continued search:

$$\max_{\tau \in \mathcal{T}} \mathbb{E}_{\pi_{\text{search}}} \left[b_\tau - \alpha C_\tau \right],$$

where b_τ is the best commit-now reward among opened/scored candidates by stopping time τ, C_τ is the number of environment steps, and $\alpha > 0$ penalizes additional search. In the WebShop domain, the selected candidate is a product, and in Paper Search, the candidate is the best opened paper found so far.

3.2 Current best reward

Let $\mathcal{I}_t$ be the set of candidate impressions observed by time t, and let $\mathcal{O}_t$ be the set of candidate pages opened and scored by time t. The current best reward is

$$b_t = \max\big(\{0\} \cup \{u(v) : v \in \mathcal{O}_t\}\big),$$

where $u(v)$ is the utility of committing to the candidate on an opened page. The results of each search can update $\mathcal{I}_t$ and the agent's context, but they do not change b_t until a candidate's page is opened.

In WebShop, $v = (x_v, o_v)$ is a product page and $u(v) = r^{\text{buy}}(x_v, o_v)$ is the reward of clicking `buy now` on the opened product as defined in the WebShop environment. In Paper Search, $u(v)$ is a normalized relevance score for the opened paper where the relevant papers rated are in the highest reward level, and the unrated papers receive a deterministic title/abstract value based on relevant.

3.3 Decision points and goal-disjoint splits

Decision points are specific search model trajectory steps where the search model sees a previously unseen candidate, opens a new candidate page, or is the last state of a trajectory. We store the goal, step, prior actions, current observation, a summary of the candidates seen along with the candidates that were opened, and the best seen reward so far b_t in each decision point.

These decision points are then split into train, development, and test sets. It is ensured that all decision points from the same trajectory are in the same split set. This is to prevent any data leakage that would give our trained models an unfair advantage.

3.4 Remaining-upside target

We forecast the remaining improvement in reward we can get by continuing the search, at each decision point. At decision point t, let s_t be the current

search state, $b_t \in [0, 1]$ be the best reward we have seen so far, and H_t be the maximum steps we can take. We define the remaining upside for a continuation w as:

$$\Delta_t(\omega) = \max_{0 \le h \le H_t} b_{t+h}(\omega) - b_t.$$

The conditional expected upside below is used for the stopping and allocation policies.

$$\mu_t = \mathbb{E}[\Delta_t(\omega) \mid s_t]\,.$$

We replay the trajectory prefix up to each decision point and sample K continuations. This is used to build our supervised labels.

$$\Delta_t^{(k)} = \Delta_t(\omega_k), \qquad k = 1, \ldots, K.$$

4 Method

4.1 Decision-point data

We ensure the experiments isolate only the modifications to the forecaster. We do this by maintaining the same search policy throughout generating the data. We use `Qwen/Qwen2.5-7B-Instruct` (Qwen et al., 2025) and prompt it to choose an environment action at each step. We train the forecasting model with the decision points we get from the trajectories.

4.2 Continuation labels

We replay each decision point's previous actions to reconstruct the state and then sample $K = 6$ continuations from that state. For each continuation k, $\Delta_t^{(k)}$ represents the increase from b_t to the best seen reward from that continuation.

4.3 Hurdle-Beta (ZOIB) regression head

The forecasting model is queried at each decision point and generates a set of parameters. From each continuation, we get the overall best reward in that trajectory, which gives us how much the reward has changed from the best seen at the decision point. This improvement can only be between 0 and 1. Many of the continuations also give us no improvement. To best model this, we use a discrete mass at 0, a discrete mass at 1, and a Beta distribution over $(0, 1)$:

$$\widetilde{\Delta}_t = \begin{cases} 0, & \text{with probability } \hat{p}_{0,t}, \\ 1, & \text{with probability } \hat{p}_{1,t}, \\ X_t, & \text{with probability } \hat{p}_{\text{int},t}, \end{cases}$$

```
INPUT
  Fixed search policy and the goal-disjoint splits

TRAIN THE FORECASTER
  1. Generate one trajectory for each training task.
  2. At each decision point:
       a. Replay previous actions to re-build the state.
       b. Sample K continuations, with the last step being
      H at most.
       c. Label each by its improvement over the
          current best seen reward.
  3. Render the state and fit the head to its K labels.
  4. Select the checkpoint on development data set.

EVALUATE ON FIXED TEST STREAMS
  Multi-thread allocation:
    Advance the stream with the greatest predicted mean
    remaining reward improvement until budget is reached.
  Single-trajectory stopping:
    Replay each stream independently and stop when the
      chosen
    predicted-gain-versus-cost rule is met.
```

Listing 1: Full forecasting and fixed-replay pipeline. Both evaluations use the same forecaster and held-out fixed trajectories.

where

$$\hat{p}_{\text{int},t} = 1 - \hat{p}_{0,t} - \hat{p}_{1,t}, \qquad X_t \sim \text{Beta}\big(\hat{m}_t^+ \hat{\kappa}_t^+, (1 - \hat{m}_t^+)\hat{\kappa}_t^+\big)$$

$\hat{m}_t^+$ represents the mean interior improvement and $\hat{\kappa}_t^+$ represents how concentrated the interior distribution is around that mean.

The allocation and stopping policies downstream use

$$\hat{\mu}_t = \mathbb{E}[\widetilde{\Delta}_t] = \hat{p}_{1,t} + \hat{p}_{\text{int},t}\hat{m}_t^+$$

which represents the average estimate of how much more reward will be found by the search model if we continue from the step we are currently at.

However, at each decision point, the amount the reward improves by cannot be greater than $1 - b_t$. This is because the current best reward is already b_t and the reward cannot be greater than 1. To take this into consideration, we evaluate a support-aware variant as well. This still predicts the same distribution, but bounded.

At each decision point, the forecasting model, which comprises of an LLM backbone (Hu et al., 2022) with LoRA adapters and a regression head, predicts the parameters of the distribution. The training uses the per-sample hurdle-Beta negative log likelihood along with auxiliary expected value and event-Brier losses. Evaluation uses only the scalar $\hat{\mu}_t$ that is reduced from the full distribution to rank or stop searches.

5 Experiments

5.1 Domains

We experiment and evaluate with two domains: WebShop and Paper Search. We construct Paper Search ourselves using LitSearch and BEIR scientific IR datasets. Paper utility, which we use as part of the reward for Paper Search, is a relevance score between the query and the title and abstract of a given paper. For WebShop, we use the buy-now reward and goals. Table 1 has the train/dev/test scale we used for the forecaster.

We split the datasets for each domain in a task-disjoint way. The quantity of data for each domain and each set is in Table 1.

5.2 Models

Our baselines are a prompted base LLM, the same model as the LLM used for search; a step-only ridge setting, which predicts the remaining improvement in reward there is left by only being informed of the position in the trajectory; a feature-only setting where information about the current best seen reward is also present; and a reservation control arm inspired by the Pandora's Box problem, where we examine the training continuations and group those into bins based on the step in the trajectory and the best seen reward so far. Appendix E provides more information on its implementation.

We use these baselines to evaluate four experimental arms: hurdle-beta, support-aware hurdle-beta, a scalar mean-only head, and a Gaussian head.

5.3 Two Fixed-Replay Evaluation Protocols

Multi-thread budgeted expansion. In this task, we have multiple searches running at once that have a combined total budget. Each of the threads begin at the first opened candidate page, and at each step, we move the selected thread to its next decision point. Which thread is selected is based on which one has the highest $\hat{\mu}_t$, or predicted remaining improvement in reward. The forecast then ranks all the threads, but does not decide whether a trajectory should be stopped. The step cost is the change in mean final steps. We evaluate the prompted LLM baseline, the trained heads, and the non-LLM baselines. We compare these against an oracle that knows every reward and calculates the best aggregate reward at each step budget. Figures 2 and 3 show the same runs as oracle gap and absolute reward, respectively.

Single-trajectory stopping replay. We consider each search independently in this task. At each decision point, we either stop and commit to the best seen reward or continue along the trajectory.

Let $H_t = max(1, H - t + 1)$ be the number of search steps left at step t, including the current step. Let $\bar{h}_t = (H_t + 1)/2$ be the estimated average number of search steps that will be taken if the trajectory is continued from this step. The EV-cost rule stops when

$$\hat{\mu}_t - c_{\text{step}}\bar{h}_t \leq 0,$$

and the risk-penalized rule stops when

$$\hat{\mu}_t - \lambda_{\text{risk}}\hat{\sigma}_t - c_{\text{step}}\bar{h}_t \leq 0.$$

We conduct a sweep of the same c_{step} and λ_{risk} values for each forecast. We use this for a reward-step frontier plot shown in Figure 4.

6 Results

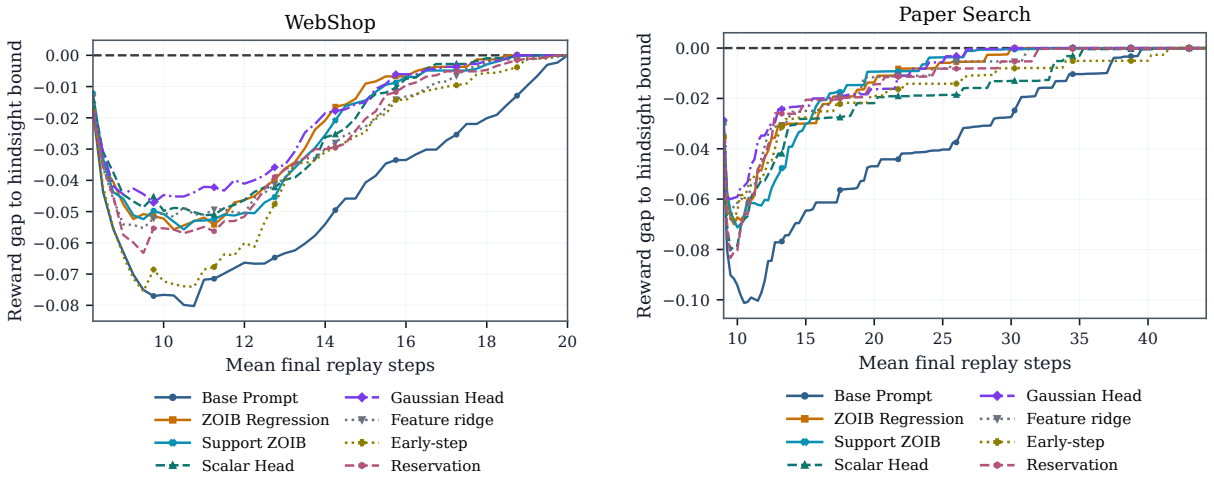
Our results can be broken down into three main parts: Can the forecasting model identify which search threads are worth spending more search budget on? Can the forecasting model identify when a search thread should stop searching and commit to the best candidate seen so far? Is the value coming from our methods or other features like the fact that earlier steps usually have more value than later ones?

6.1 Multi-Thread Budgeted Expansion

Training a model to forecast the remaining improvement in reward performs much better than prompting an LLM. Figure 2 shows us this as the curves closer to the hindsight line are better. Table 2 further shows us that hurdle-beta, scalar, and Gaussian heads all improve allocation AUC over our prompted baseline in both domains. Even though the Gaussian results are the highest, the difference between all of the trained heads are not significant enough to separate them. We can also see the results on an absolute reward scale as shown in figure 3. The primary result here is that learning the remaining improvement in reward is useful while the exact regression head does not matter as much. It also shows that the step in the search along with other basic trajectory information is especially important to predicting the remaining improvement in reward.

Domain	Task unit	Tasks	Train	Dev	Test	Commit-now utility
WebShop	Shopping goal	4,155	19,967	1,997	2,007	WebShop buy-now reward for opened product/options
Paper Search	Literature query	3,647	19,967	1,997	2,007	Qrels plus title/abstract relevance for opened papers

Table 1: Training and evaluation data for the two domains. Train, development, and test columns have the number of decision-point labels with Monte Carlo continuation targets for remaining improvement in reward. Goal-disjoint splits assign every decision point from the same search task to one split.



(a) WebShop

(b) Paper Search

Figure 2: Multiple-thread budget allocation under a predicted remaining improvement of reward. At each turn, a method chooses which trajectory to move forward with. Trained forecasts allocate search better than the prompted baseline. Gaussian, hurdle-beta, scalar, and other controls are close.

Protocol	Domain	Prompt	Raw ZOIB	Scalar	Gaussian
Stopping	WebShop	0.5369	0.5419	0.5373	**0.5454**
Stopping	Paper Search	0.3640	0.3765	0.3697	**0.3795**
Raw allocation	WebShop	0.5325	0.5541	0.5537	**0.5564**
Raw allocation	Paper Search	0.3652	0.3888	0.3827	**0.3906**

Table 2: Each allocation or stopping curve is represented as one number. A higher number means better reward across the search costs. All the trained heads generally beat the prompted baseline. The differences between the trained heads are small.

6.2 Single-Trajectory Stopping Replay

The Gaussian head has the highest value on the stopping AUC and the hurdle-beta head and reservation control are close. The learned forecasts are clearly better on the Paper Search domain, whereas on the WebShop domain, the results are more nuanced as there is not as much separation in scores between the trained heads and the baselines. Results are shown in figure 4. Other simple trajectory controls are strong as well. The step-only baseline and the train-only reservation control are both competitive to the learned heads. These results also shows that the step in the trajectory is an especially meaningful signal as to when the model should stop searching.

Stopping seems to be a harder problem for this method than allocation. This is likely because a forecaster only needs to answer whether search thread A has more opportunity than search thread B for allocation. Knowing how much improvement is left in terms of reward is an added complexity. That prediction is needed for stopping as well, since questions like whether the remaining improvement in reward is worth N more steps.

6.3 Cost and Support Controls

We can also see that the Pandora-inspired control is a strong competitor, especially on WebShop. This method predicts the value of continuing the search by only using the training continuations grouped by the step in the search trajectory and the best seen reward until this point. This is not exactly a Weitzman policy since LLM actions generate future states and do not select from a fixed set of boxes.

We can draw the same conclusions from the cost-normalizing allocation as well. No single trained head uniformly is the best.

We can see the effects of the hurdle-beta model without the support-aware constraint. It assigns probabilities over the possible value in WebShop and Paper Search. The support-aware version fixes this, but does not significantly affect overall performance. Appendix E has the control details.

7 Analysis

Through these experiments, we can conclude that predicting the remaining improvement in reward is a useful signal for searching with a fixed policy.

We can see that no single trained model uniformly wins on every metric. The hurdle-beta model predicts an entire distribution including the probability of no improvement, the most possible improvement, and all improvements in between. The scalar method just predicts one number, not a distribution. The Gaussian model predicts a mean and uncertainty using a normal distribution. We can see that the hurdle-beta model is the best at ranking opportunities since the results have the strongest correlation with the targets from the Monte Carlo continuations. This means the hurdle-beta model answers the question of which search states have more possible improvements in reward remaining. The Gaussian model seems to be better at estimating that exact number, however. We can see that the Gaussian model's predicted number is closest to the oracle value on average. We can clearly separate the ability to rank search threads and the ability to estimate a scalar accurately as different success metrics. Since we want to use the outputs of these forecasts to decide whether we continue searching or stop and commit to the best candidate seen so far, the Gaussian model's strength of estimating numerically, allows it to score the best on the downstream tasks. Though it is not a significant difference compared to hurdle-beta and scalar. One aspect to consider is our actual possible remaining improvement in reward is upper bounded, and the only model that adheres to that is the support-aware hurdle-beta model.

We can also see that where the step is in the overall trajectory can play a role in performance as seen in figure 5, where the gains in reward improvement tend to be earlier on in the trajectory rather than later. However, based on comparing states only with other states that are in approximately the same stage of the trajectory, we can see that though the correlations decrease, they do not go down to zero. This means that even after reducing this bias, models like hurdle-beta can still identify which states have more possible improvement in reward left. Appendix F has the complete results.

We can say that learned forecasting models can improve upon information about the search trajectory to perform better on allocation and stopping. We can likely further improve the forecasts by using better representations of the current states and increasing the number of our Monte Carlo continuations.

8 Conclusion

We study whether we can estimate the remaining improvement of reward over the current best candidate at each decision point in a search trajectory. We found that trained models are useful for allocating a search budget among many search threads. They can also be useful for identifying whether we should continue searching or stop and commit to the best candidate seen so far. An important finding was that the information on the trajectory is already strong, but learned methods can further improve on them, particularly for allocation. These findings are useful since search agents have to manage compute and by predicting whether there is value in continuing to search is one way to do that efficiently.

Limitations

We use $K = 6, H = 60$ which can both be increased. The horizon can also be made variable to allow us to evaluate on multiple values. We also forecast remaining improvements of reward under fixed trajectories from one search model. Future work can optimize both a search policy and a forecasting method together. Our claims in this paper are for forecasting opportunity under a fixed policy.

We also assume that a reward can be assigned to

an opened candidate at each step where that happens. We use the reward defined in the WebShop in that domain and build a reward for the Paper Search domain. In systems that are deployed in the real world, a reward model or verifier must be devised.

Ethical Considerations

Though better continuation policies can prevent unnecessary compute and time, it is important to understand the forecasts in terms of tradeoff between reward and the cost of searching.

Any learned policy in the e-commerce domain can have any biases that are present in how the reward of a product is calculated or the total products present. Since we construct the rewards for the paper search domain, they should be used as a guide for how useful the paper is. Generally, in high stakes tasks, human judgment should be the final decision maker and this method should be used as a guide.

References

Anirudh Ajith, Mengzhou Xia, Alexis Chevalier, Tanya Goyal, Danqi Chen, and Tianyu Gao. 2024. Litsearch: A retrieval benchmark for scientific literature search. In *EMNLP 2024 - 2024 Conference on Empirical Methods in Natural Language Processing, Proceedings of the Conference*, EMNLP 2024 - 2024 Conference on Empirical Methods in Natural Language Processing, Proceedings of the Conference, pages 15068–15083. Association for Computational Linguistics (ACL). Publisher Copyright: © 2024 Association for Computational Linguistics.; 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024 ; Conference date: 12-11-2024 Through 16-11-2024.

Alexia Atsidakou, Constantine Caramanis, Evangelia Gergatsouli, Orestis Papadigenopoulos, and Christos Tzamos. 2024. Contextual pandora's box. In *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence and Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence and Fourteenth Symposium on Educational Advances in Artificial Intelligence*, AAAI'24/IAAI'24/EAAI'24. AAAI Press.

Glenn W Brier. 1950. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3.

Shuchi Chawla, Evangelia Gergatsouli, Jeremy McMahan, and Christos Tzamos. 2021. Approximating pandora's box with correlations. In *International Workshop and International Workshop on Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques*.

Mehul Damani, Isha Puri, Stewart Slocum, Idan Shenfeld, Leshem Choshen, Yoon Kim, and Jacob Andreas. 2026. Beyond binary rewards: Training LMs to reason about their uncertainty. In *The Fourteenth International Conference on Learning Representations*.

Thomas S. Ferguson. 1989. Who solved the secretary problem? *Statistical Science*, 4(3):282–289.

P. Frazier, Warrren B Powell, and Savas Dayanik. 2009. The knowledge-gradient policy for correlated normal beliefs. *INFORMS J. Comput.*, 21:599–613.

Peter I. Frazier, Warren B. Powell, and Savas Dayanik. 2008. A knowledge-gradient policy for sequential information collection. *SIAM Journal on Control and Optimization*, 47(5):2410–2439.

Evangelia Gergatsouli and Christos Tzamos. 2023. Weitzman's rule for pandora's box with correlations. In *Thirty-seventh Conference on Neural Information Processing Systems*.

Tilmann Gneiting and Adrian E Raftery. 2007. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning - Volume 70*, ICML'17, page 1321–1330. JMLR.org.

Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

T. Tony Ke and J. Miguel Villas-Boas. 2019. Optimal learning before choice. *Journal of Economic Theory*, 180:383–437.

Jing Yu Koh, Stephen Marcus McAleer, Daniel Fried, and Ruslan Salakhutdinov. 2025. Tree search for language model agents. *Transactions on Machine Learning Research*.

Grace Liu, Yuxiao Qu, Jeff Schneider, Aarti Singh, and Aviral Kumar. 2025. CaRT: Teaching LLM agents to know when they know enough. In *First Workshop on Multi-Turn Interactions in Large Language Models*.

Tengxiao Liu, Zifeng Wang, Jin Miao, I-Hung Hsu, Jun Yan, Jiefeng Chen, Rujun Han, Fangyuan Xu, Yanfei Chen, Ke Jiang, Samira Daruki, Yi Liang, William Yang Wang, Tomas Pfister, and Chen-Yu Lee. 2026. Budget-aware tool use enables effective agent scaling. In *Third Conference on Language Modeling*.

William S. Lovejoy. 1991. A survey of algorithmic methods for partially observed markov decision processes. *Ann. Oper. Res.*, 28(1–4):47–66.

Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Ouyang Long, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, Xu Jiang, Karl Cobbe, Tyna Eloundou, Gretchen Krueger, Kevin Button, Matthew Knight, Benjamin Chess, and John Schulman. 2021. Webgpt: Browser-assisted question-answering with human feedback. *ArXiv*, abs/2112.09332.

Young-Jin Park, Kristjan Greenewald, Kaveh Alim, Hao Wang, and Navid Azizan. 2025. Know what you don't know: Uncertainty calibration of process reward models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, and 25 others. 2025. Qwen2.5 technical report. *Preprint*, arXiv:2412.15115.

Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: Bm25 and beyond. *Found. Trends Inf. Retr.*, 3(4):333–389.

C. Nicolò De Sabbata, Theodore Sumers, and Thomas L. Griffiths. 2024. Rational metareasoning for large language models. In *NeurIPS 2024 Workshop on Behavioral Machine Learning*.

Richard D. Smallwood and Edward J. Sondik. 1973. The optimal control of partially observable markov processes over a finite horizon. *Operations Research*, 21(5):1071–1088.

Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*.

Martin L. Weitzman. 1979. Optimal search for the best alternative. *Econometrica*, 47(3):641–654.

Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. Webshop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems*, volume 35, pages 20744–20757. Curran Associates, Inc.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. Publisher Copyright: © 2023 11th International Conference on Learning Representations, ICLR 2023. All rights reserved.; 11th International Conference on Learning Representations, ICLR 2023 ; Conference date: 01-05-2023 Through 05-05-2023.

Yuanzhao Zhai, Tingkai Yang, Kele Xu, Dawei Feng, Cheng Yang, Bo Ding, and Huaimin Wang. 2025. Enhancing decision-making for llm agents via step-level q-value models. In *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence and Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence and Fifteenth Symposium on Educational Advances in Artificial Intelligence*, AAAI'25/IAAI'25/EAAI'25. AAAI Press.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. Webarena: A realistic web environment for building autonomous agents. In *The Twelfth International Conference on Learning Representations*.

A Domain Construction Details

A.1 Decision-Point Generation

Decision points are generated once for each goal or query. We use the same process to generate trajectories and identify which steps within those trajectories should be decision points. We maintain a horizon $H = 60$. We use seed 123 as well. The generator updates the candidates observed in the set that stores that, what candidates were opened, and what is the best reward seen until this point. The data is split into train, development, and test sets in a goal disjoint way to prevent goals being split across sets and to prevent data leakage. These sets are made before labeling. We score product page states for the WebShop domain with the provided buy-now reward. We keep the option dictionary currently stored in the environment as is.

A.2 Paper Search Environment

The Paper Search environment is created from LitSearch, SciDocs, SciFact, NFCorpus, and TREC-COVID. We have 3,647 queries, 269,988 papers, and 81,722 query relevance rows.

The environment is constructed similar to how the WebShop environment is. We preserve the same search and click actions. In the Paper Search domain, the actions include searching that is based on the query text and the pages of results, clicking into a paper, moving back and forth across pages, clicking back to a results page, and stopping. Similar to WebShop, there are multiple levels of queries from broader ones like a short keyword query to a longer and even original query.

The retrieval is built using BM25-style scoring with the title and the abstract of the papers (Robertson and Zaragoza, 2009). We use a deterministic perturbation over the BM25 pool that does not consider reward. We do not use query relevance for

ranking since only an opened paper can be used as a best candidate seen so far. The `gold_hit@stop` metric is also stored to make the value of the query relevance interpretable by being able to see how many times the final chosen paper in each trajectory relates to the query relevance of the paper.

A reward is assigned to each paper that is opened. The rewards are bounded in $[0, 1]$ and a paper with the highest positive query relevance score gets a reward of 1. Other papers that still have a positive query relevance score get a reward that is between 0.90 and 1. If a paper is not judged, it can get a reward of 0.75 at most.

The relevance of the not judged paper to the query is combined with the similarity of the paper to the most similar paper that has a positive query relevance score. The weights used are 0.40 and 0.60. The relevance between the query and the paper averaged the query-normalized BM25 and TF-IDF cosine similarity over the titles and abstracts. Only the relevance between the query and the paper is used when there are no other papers with a positive query relevance score.

By constructing the reward this way, the papers with a positive query relevance score will always score higher than ones without. But, every paper that is not judged still gets some credit. This mimics WebShop's partial-match reward.

B Label and Target Details

Monte Carlo continuations use a temperature of 0.7 and a top-p of 0.9. We use a horizon, or maximum number of steps of $H = 60$. The labels generated for each continuation k is the improvement in reward from the best reward until that point:

$$\Delta_t^{(k)} = \max_{0 \le h \le H_t} b_{t+h}^{(k)} - b_t,$$

Budget allocation and expected-value stopping use $\hat{\mu}_t$. Risk-penalized stopping also uses the standard deviation of the same predicted distribution.

C Prompt and Parsing Details

The forecasting model gets the task, a summary of candidates seen, a summary of candidates opened, the context of the current decision point, and the best reward seen until this point.

```
You are forecasting the upside from continuing
    to search for a search goal.

Search Goal: {goal}

Candidates seen so far:
{seen_candidates}

Current decision point:
{state_context}

Current best reward from visited candidate pages
     seen so far: {best_reward_seen}

Task:
- Forecast the remaining upside from continuing
    the search.
- Let Delta = max_future(best reward seen so far
    ) - best reward seen now.
- Output a hurdle-Beta remaining-upside forecast.


Output exactly:
<think>brief reasoning</think>
<delta_zero_prob>p0</delta_zero_prob>
<delta_one_prob>p1</delta_one_prob>
<delta_pos_mean>m_plus</delta_pos_mean>
<delta_pos_concentration>k_plus</
    delta_pos_concentration>
<analysis>brief justification</analysis>
```

The last instance of each tag is extracted by the parser. Hurdle-Beta outputs are valid only if $p_0 \in [0, 1]$, $p_1 \in [0, 1]$, $p_0 + p_1 \le 1$, $m^+ \in (0, 1)$, and $\kappa^+ > 0$. Forecasts that are invalid do not get any remaining reward improvement value. The regression head forecaster also does not need the parser since its outputs are scalars.

D Training and Evaluation Details

We use `Qwen/Qwen2.5-7B-Instruct` (Qwen et al., 2025) as the search model for all tasks. The baseline prompted forecaster is asked to output a hurdle-Beta forecast.

Experimental details are outlined in section 5. The search-value profile in Figure 5 uses the same trajectories, six ten-step bins over $H = 60$, and finite look-ahead budgets of 5, 10, 20, and all remaining trajectory steps.

E Reservation-Value and Support Controls

The Pandora-inspired control estimates the continuation-gain distribution from training continuations after binning states by checkpoint step and current best reward, then applies the empirical mean to held-out states. For nonnegative gain D, the $z = 0$ boundary of the reservation equation $\mathbb{E}[(D - z)_+] = c$ is $\mathbb{E}[D] > c$, so we sweep the same continuation costs as for the learned stopping rules. This is a train-only reservation-value control and not a claim of Weitzman optimality. A more faithful Weitzman rule uses fixed candidates

Held-out Paper Search state	Mean reward	Qrel-positive hit	Top-grade qrel hit
First schedulable decision point	0.287	83 / 360 (23.1%)	81 / 360 (22.5%)
Full fixed trajectory	0.403	124 / 360 (34.4%)	124 / 360 (34.4%)

Table 3: Paper Search query relevance metric on held-out test trajectories. A reward of at least 0.90 is a paper with a positive query relevance score.

and one-time inspection costs, whereas a search thread can be expanded many times and each LLM generation changes what states will be seen in the future.

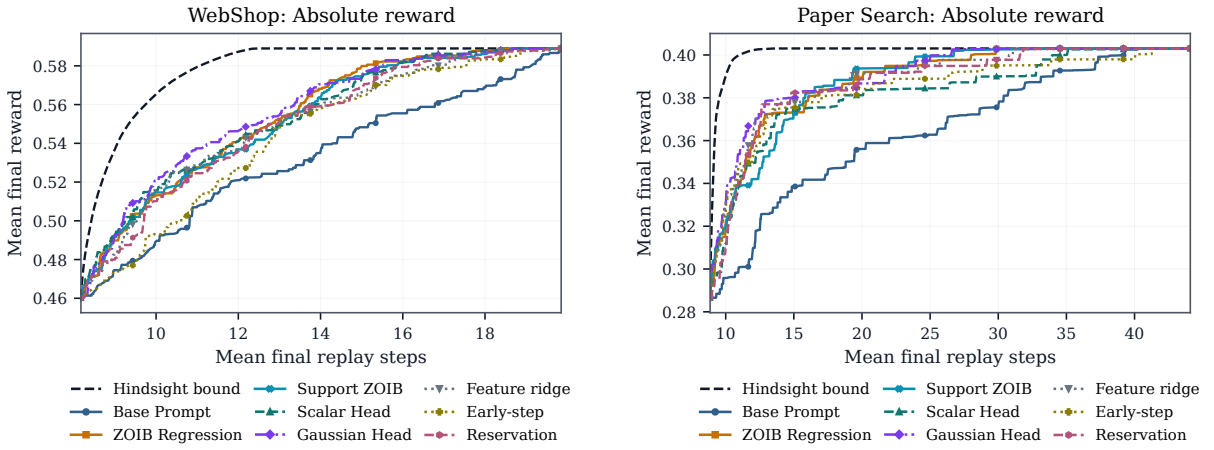
Run settings needed to reproduce the final results are given in Table 6. Table 7 reconciles the source task counts, labeled decision points, complete replay streams, and prompted-cache records. Table 8 reports prompt parsing on all test labels and a retry-on-malformed-output sensitivity. The review artifact includes code, goal-disjoint splits, held-out trajectory/checkpoint streams, and continuation labels. Its README gives one manifest-driven path through the pipeline: `scripts/verify_emnlp_rebuttal_manifest.py` verifies frozen inputs; `scripts/submit_emnlp_rebuttal_v1_stack.py` launches training, prediction, controls, and both replay protocols; `scripts/build_budgeted_expansion_frontier.py` builds the multi-thread curves; `scripts/sweep_stop_policies_from_prediction_caches.py` and `scripts/summarize_stop_policy_sweep.py` build the single-trajectory frontiers; and `scripts/audit_emnlp_rebuttal_results.py` verifies the resulting artifacts against the same manifest.

F Forecast Diagnostics

Table 9 evaluates predicted expected upside against the mean of the six Monte Carlo continuation labels at each held-out decision point. This is distinct from correlating with one realized replay trajectory. Global ρ_{MC} pools decision points; within-step ρ_{MC} is computed inside 10-step bins and averaged by bin size, reducing the contribution of broad trajectory position. These expected-value diagnostics evaluate the forecast used by the scheduler, not the full positive-tail distribution.

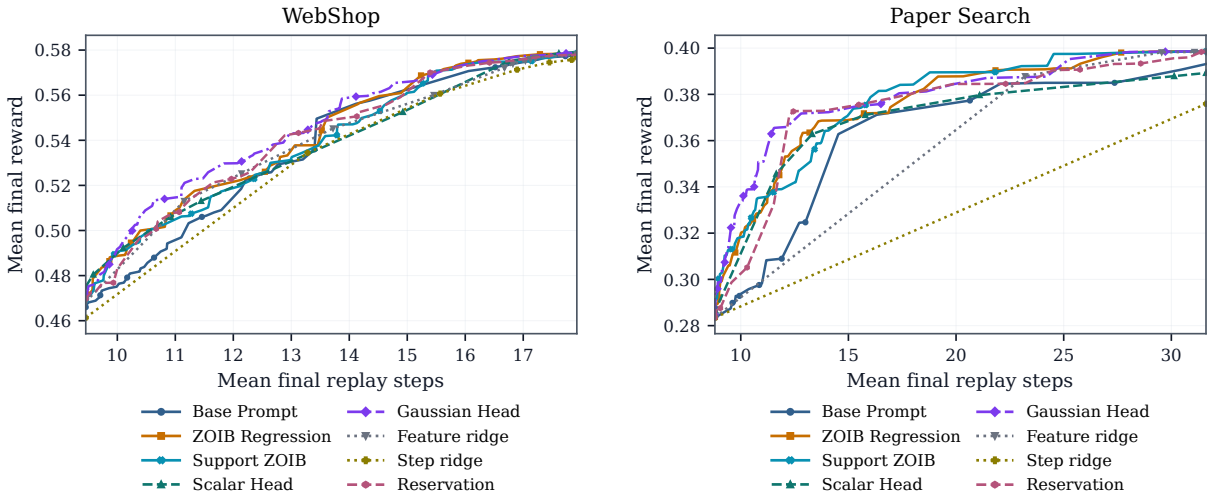
Method	Stopping AUC		Cost-normalized allocation AUC	
	WebShop	Paper Search	WebShop	Paper Search
Pandora-inspired control	0.5414	0.3746	0.5597	0.3900
Raw hurdle-Beta	0.5419	0.3765	0.5625	0.3911
Support-aware hurdle-Beta	0.5391	0.3774	0.5610	**0.3918**
Gaussian	**0.5454**	**0.3795**	**0.5631**	0.3915

Table 4: Stronger stopping, cost-normalized allocation, and support controls.



(a) WebShop (b) Paper Search

Figure 3: Absolute-reward framing of the same multiple-thread runs as Figure 2, with the fixed-replay hindsight oracle shown.



(a) WebShop (b) Paper Search

Figure 4: Single-trajectory stopping frontiers on held-out trajectories. Each curve is a Pareto envelope over the corresponding stop-cost sweep. The step-only and feature-only controls test whether trajectory metadata explains the learned forecasts. The train-only reservation controls test the training continuation reward improvements.

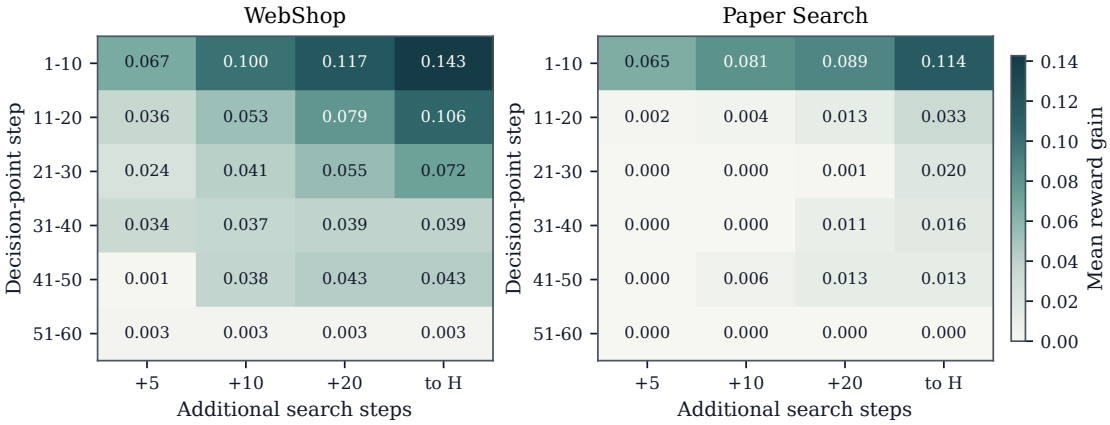


Figure 5: Where the future reward remains. Each cell asks from this point in the trajectory, how much additional reward is found after each step count.

	WebShop			Paper Search		
Method	Global ρ_{MC}	Within-step ρ_{MC}	Stop AUC	Global ρ_{MC}	Within-step ρ_{MC}	Stop AUC
Base Prompt	0.105	-0.007	0.5369	0.091	-0.063	0.3640
Raw ZOIB	**0.854**	**0.747**	**0.5419**	**0.689**	**0.458**	**0.3765**
Step-only	0.560	0.279	0.5312	0.466	0.134	0.3297
Feature-only	0.803	0.655	0.5385	0.590	0.330	0.3573
Reservation	0.795	0.661	0.5414	0.580	0.291	0.3746

Table 5: Trajectory-position and simple-control diagnostics. ρ_{MC}, rank correlation score, ranks predicted upside against the mean of the six Monte Carlo labels. Within-step correlation is calculated inside 10-step bins and averaged by bin size. Stop AUC uses the held-out replay trajectories in Figure 4.

Component	Setting
Search and labels	`Qwen/Qwen2.5-7B-Instruct` is used as the search model. One source trajectory per task is generated with temperature 0 and top-$p = 1$. Monte Carlo continuations use the same model and action prompt with temperature .7 and top-$p = .9$. We use horizon $H = 60$ and $K = 6$ continuation targets per decision point.
Data and inputs	We use task-disjoint train/dev/test splits with seed 123. Rewards and gains are within $[0, 1]$. All methods receive the same decision points, labels, and splits. The trained heads use the final hidden state of the LLM backbone.
Forecasting heads	ZOIB predicts probability mass at 0 and 1 and a Beta distribution over interior gains. Support-aware ZOIB predicts $\Delta/(1 - b_t)$ and rescales by $1 - b_t$. Scalar predicts the mean gain directly. Gaussian predicts a mean and standard deviation, with a standard-deviation floor of .01.
Training objectives	ZOIB uses endpoint-aware negative log-likelihood, expected-value Huber loss with weight .2, and event-Brier loss with weight .1. Scalar uses Huber loss on the mean of the six continuation targets. Gaussian uses the exact six-target Gaussian negative log-likelihood.
Optimization	We train for three epochs with seed 42, configured batch size 1, and gradient accumulation 16. LoRA uses rank 16, alpha 32, dropout .05, and the `q/k/v/o/gate/up/down_proj` modules. The learning rate is 10^{-4} except for Paper Search Gaussian, which uses 10^{-5}. Scalar is selected by dev MAE. ZOIB and Gaussian are selected by their dev objectives.
Prompt baseline	The same Qwen model produces a ZOIB forecast at temperature 0, with context length 4096 and at most 1024 new tokens. Forecast metrics use valid numeric parses. Invalid forecasts get zero allocation priority and cannot trigger a stopping action.
Evaluation	Every method uses the same aligned fixed-replay trajectories. Stopping uses one shared cost/risk value sweep. AUC is normalized over each domain's shared observed support.
Reproduction	Run `download-data`, `validate-data`, `prepare-model`, `experiment -scheduler slurm -start-from labels -run-root runs/paper`, `summarize -run-root runs/paper`, and `paper` through `python -m opportunity_forecasting`. Frozen settings are in `configs/paper.json`.
Software	Python 3.11.14. Core runs use PyTorch 2.5.1, Transformers 4.48.3, and vLLM 0.7.3. WebShop trajectory and label generation use PyTorch 2.6.0, Transformers 4.51.3, and vLLM 0.8.5.post1. Full package locks are released.

Table 6: Settings to run the code for the reported experiments. The trajectories and Monte Carlo labels use the same fixed search model.

Domain	Source tasks	Test labels	Aligned test goals	Possible allocation streams	Prompt test predictions
WebShop	4,155	2,007	371	365	2,007
Paper Search	3,647	2,007	364	360	2,007

Table 7: Source tasks are the available WebShop goals or Paper Search queries. Test labels are the Monte Carlo-labeled decision points in Table 1. Aligned test goals have common decision point, prediction, and trajectories across all the methods. The possible allocation threads start with an opened candidate and go through the multiple thread allocation runs.8.

	Valid parse rate		MC-label MAE	
Domain	Original	Retry invalid	Original	Retry invalid
WebShop	0.815	0.935	0.2467	0.3021
Paper Search	0.955	0.989	0.1738	0.2011

Table 8: Prompt parse-health sensitivity on the test labels. Retrying only malformed numeric outputs improves parse coverage but worsens MC-label MAE. MAE is calculated over valid numeric parses for each variation. Parse coverage is shown separately. Invalid forecasts get zero scheduling priority in allocation. In stopping, they cannot trigger a stop.

Domain	Method	MAE ↓	Global ρ_{MC} ↑	Within-step ρ_{MC} ↑
WebShop	Base Prompt	0.2467	0.105	−0.007
WebShop	Raw ZOIB	0.1206	**0.854**	**0.747**
WebShop	Scalar	**0.1153**	0.842	0.730
WebShop	Gaussian	0.1162	0.840	0.718
Paper Search	Base Prompt	0.1738	0.091	−0.063
Paper Search	Raw ZOIB	0.0583	**0.689**	**0.458**
Paper Search	Scalar	0.0570	0.467	0.155
Paper Search	Gaussian	**0.0490**	0.655	0.419

Table 9: Forecast quality against the mean of the six Monte Carlo continuation labels. Global correlation pools decision points. Within-step correlation is computed inside 10-step bins and averaged by bin size. The Base Prompt metrics only use valid numeric parses.